

Cross Validation Fall 2026

Chris Mecklin

Cross-Validation

Validation Set Approach

When we build a “rule” to make predictions, whether we use a traditional approach like a linear model from the “data modeling” culture or an approach such as CART from the “algorithmic” culture, we wish to know if the rule we have will do a good job in predicting future outcomes. The **validation set** approach considers this problem by dividing a data set into two random pieces, typically (but not always) of the same size. One of this pieces is called the **training set** and is used to build a model or prediction rule, and the other piece is held out and used afterwards as the **test set** for validating the model. The hope is that your model/rule will accurately predict values from the test set, yielding a low *MSE* (or a high accuracy/low error rate in the case of classification).

For example, the R package MASS has two data sets called `Pima.tr` and `Pima.te` that we have already considered. It turns out that the complete data set, dealing with incidence of Type II diabetes among women on the Pima reservation, had $n = 532$ cases, and `Pima.tr` is a random selected set of $n = 200$ of these cases. The `Pima.te` set contains the remaining $n = 332$ cases.

We will use `Pima.tr` as the training set and `Pima.te` as the test set for the model

$$\text{bmi} \sim \text{age} + \text{bp} + \text{npreg}$$

where we try to predict the woman’s body mass index based on her age, her diastolic blood pressure, and the number of pregnancies that she has had. I’ll use linear regression and CART as a comparison of the two cultures; often in the machine learning world people will try many methods as accurate prediction is the main goal, with interpretation or statistical inference as lesser goals (or possibly not cared about at all).

```
require(tidyverse)
require(MASS)
data(Pima.tr)
data(Pima.te)
# build the linear regression "rule" with the training set
bmi.linreg <- lm(bmi ~ age + bp + npreg, data=Pima.tr)
summary(bmi.linreg)
```

Call:

```
lm(formula = bmi ~ age + bp + npreg, data = Pima.tr)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-14.0049	-4.8014	0.1607	3.7931	15.0114

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	22.89520	2.71260	8.440	6.94e-15 ***
age	0.03807	0.05081	0.749	0.4546
bp	0.11847	0.04020	2.947	0.0036 **
npreg	-0.06994	0.15755	-0.444	0.6576

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.99 on 196 degrees of freedom

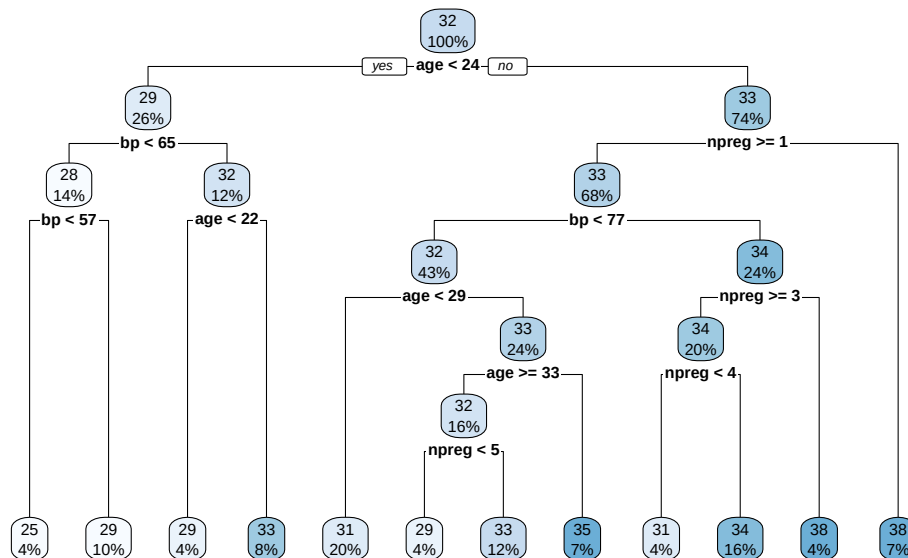
Multiple R-squared: 0.05973, Adjusted R-squared: 0.04534

F-statistic: 4.15 on 3 and 196 DF, p-value: 0.00704

```
# use it to make predictions from the test set
y <- Pima.te$bmi
yhat.linreg <- predict(bmi.linreg, newdata=Pima.te)
# compute the MSE
n <- nrow(Pima.te)
MSE.linreg <- sum((y-yhat.linreg)^2)/n
MSE.linreg
```

[1] 48.58372

```
# build a regression tree (CART) using recursive partitioning
require(rpart)
require(rpart.plot)
bmi.cart <- rpart(bmi ~ age + bp + npreg,data=Pima.tr)
rpart.plot(bmi.cart)
```



```
# use it to make predictions from the test set
y <- Pima.te$bmi
yhat.cart <- predict(bmi.cart,newdata=Pima.te)
# compute the MSE
MSE.cart <- sum((y-yhat.cart)^2)/n
MSE.cart
```

[1] 47.01216

Notice the $MSE = 48.58372$ when I used multiple linear regression, and it is $MSE = 47.01216$ when I used a regression tree (CART). Since lower values of MSE are preferred, the CART model seems a good choice.

There are several flaws to this approach, including:

1. Maybe we don't have enough data to be able to divide the data set into two pieces of a reasonable size.
2. Statistical methods tend to "perform worse" (large residuals in regression, errors in classification, etc.) with smaller data sets, and the validation set approach by definition reduces our sample size.

3. There can be a lot of variance in our results based on what portion of the original data set is randomly allocated to the training set versus the test set.

Leave-One-Out Cross-Validation (LOOCV)

For the reasons given above, it is often desirable to avoid splitting our data set into two pieces and instead to use each of the cases in our data set in both the training (i.e. building the model or rule) and testing (i.e. making predictions, validating if these predictions are decent) portions. **Cross-validation** is an approach that tried to compute this error without needing to use precious data to form a validation set with a training set and separate testing set. Clever methodology for accomplishing this task dates back to the 1970s, and recent interest in computer-intensive prediction algorithms, including methods of machine learning, reawakened interest in cross-validation. As noted by Bradley Efron and Trevor Hastie:

cross-validation is a *strongly* frequentist procedure.

An attractive approach, which will remind you of the **jackknife**, is the **Leave-One-Out Cross-Validation (LOOCV)** approach.

In the LOOCV approach, each individual case takes its turn being the test set for model validation, with the other $n - 1$ points serving as the training set. For example, if we leave out point $i = 1$, then the model/rule is built using points $i = 2, 3, \dots, n$. Then a prediction $\hat{Y}_{(i)}$ is made for point i using that rule, and the mean square error $MSE_i = (Y_i - \hat{Y}_{(i)})^2$ is computed.

This is repeated n times, with every point taking its turn being “left out”. Once this is done, the LOOCV estimate of MSE is computed as the average of all MSE_i .

$$CV_{(n)} = \frac{\sum_{i=1}^n MSE_i}{n}$$

An amazing fact is that for the special case of linear regression, this turns out to be no more computationally expensive than fitting a single regression model! It is possible to compute the cross-validation MSE with the formula

$$CV_n = \frac{1}{n} \sum_{i=1}^n \left(\frac{Y_i - \hat{Y}_i}{1 - h_i} \right)^2$$

where h_i is the **leverage** of data point i in the regression model, which can be computed with

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_{j=1}^n (x_j - \bar{X})^2}$$

for simple linear regression. The leverage h_i takes on a value from $1/n$ to 1, increasing as x_i is further from $\bar{x}$. These “high leverage” points have a greater impact on the estimated

regression model and are thus weighted more heavily in computing the cross-validated MSE . (do we want to find the theory and work through why this works?)

For multiple regression, the leverages are computed with the matrix equation

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

where

$$\hat{\mathbf{Y}} = \mathbf{H}\mathbf{Y}$$

k -Fold Cross-Validation (kFCV)

Unfortunately methods other than linear regression don't lead to such a computationally convenient result, and thus LOOCV can be extremely time-consuming, as we are forced to refit the model n times. Thus, it is common to instead use what is known as **k -fold cross-validation**. In $kFCV$, the data set is randomly divided into k groups ("folds") of approximately equal size. Let's take $k = 10$, a very common choice for the number of folds.

Instead of refitting the model n times, we will refit the model k times. The first time, we hold out the first fold (the "test" fold), fit/train the model with the other $k - 1$ folds, and then use the test fold to make predictions and compute the MSE using all points in that fold. We repeat this, with each fold taking its turn as the "test" fold. Finally, an estimate of the MSE is computed:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i$$

In addition to less computational time, there is also a trade-off between **bias** and **variance** which makes k -fold CV attractive.

In the validation set approach, where we divide the data set into 2 roughly equal pieces (which is essentially $k = 2$ -fold cross-validation), we will have bias since we will overestimate the MSE or test error rate.

Conversely, in leave-one-out cross-validation (which is n -fold), we will have an approximately unbiased estimate of MSE or test error rate, but at the price of very high variance.

So, if we pick some value of k between 2 and n , we can lower this variance, at the price of some bias. Empirically, $k = 5$ and $k = 10$ have been found to be good choices.

Cross-Validation for Classification Problems

When we have a classification problem (i.e. the response variable Y is categorical), then instead of computing a cross-validation estimate of MSE , we instead will compute a estimate of the error rate (i.e. how often values are misclassified). For LOOCV, we get

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n Err_i$$

where

$$Err_i = I(y_i \neq \hat{y}_i)$$

Basically, the indicator function is used to set Err_i equal to 1 when the point is misclassified and equal to 0 when it is correctly classified. The same idea is used if you use k -fold CV instead.

Confusion Matrix Metrics

In the special case of a binary response variable (such as either having or not having Type II diabetes), a 2×2 **confusion matrix** which tabulates the actual values of the binary response variable Y with the classifications $\hat{Y}$ from the training rule is obtained. Many different statistics can be computed from a confusion matrix.

An example is given below, where the confusion matrix is computed for a logistic regression model that predicts whether a patient has or doesn't have Type II diabetes, using the `Pima.tr` data set for training and `Pima.te` for testing (i.e. the validation set or 2-fold approach.)

```
require(tidyverse)
require(MASS)
diabetes.model <- glm(type~.,data=Pima.tr,family=binomial(link="logit"))
yprob <- predict(diabetes.model,newdata=Pima.te,type="response")
yhat <- factor(ifelse(yprob>0.5,"Yes","No"))
y <- Pima.te$type
caret::confusionMatrix(yhat,y)
```

Confusion Matrix and Statistics

	Reference	
Prediction	No	Yes
No	200	43
Yes	23	66

Accuracy : 0.8012

95% CI : (0.7542, 0.8428)
No Information Rate : 0.6717
P-Value [Acc > NIR] : 1.116e-07

Kappa : 0.5271

McNemar's Test P-Value : 0.01935

Sensitivity : 0.8969
Specificity : 0.6055
Pos Pred Value : 0.8230
Neg Pred Value : 0.7416
Prevalence : 0.6717
Detection Rate : 0.6024
Detection Prevalence : 0.7319
Balanced Accuracy : 0.7512

'Positive' Class : No

The accuracy is merely the proportion of correct classifications, which is the sum of the main diagonal divided by the total sample size.

Here

$$Accuracy = \frac{TN + TP}{TN + TP + FN + FP} = \frac{200 + 66}{200 + 66 + 43 + 23} = \frac{266}{332} = 0.8012$$

where TN is true negatives (the upper left hand corner in this table), TP is true positives (lower right hand corner), FN is false negatives (upper right), and FP is false positives (lower left). The `confusionMatrix` function in the `caret` package orients the confusion matrix with the predictions as rows and actual truth as columns—you will often see this reversed, along with the Yes/Positive and No/Negative reversed, so be cautious!

The error rate is of course just $1 - Accuracy$, so it is 0.1988 in this matrix.

Cohen's Kappa is a common statistic in the social sciences for measuring inter-rater reliability between two raters, and is used here with one “rater” being the training rule and the other “rater” being the actual truth.

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is the observed agreement between the training rule and truth (we called it Accuracy) and p_e is the expected agreement due to random chance.

In this problem, $p_o = 0.8012$ and

$$p_e = p_{No} + p_{Yes}$$
$$p_{No} = \frac{TN + FN}{n} \times \frac{TN + FP}{n} = \frac{243}{332} \times \frac{223}{332} = 0.4916$$
$$p_{Yes} = \frac{TP + FP}{n} \times \frac{TP + FN}{n} = \frac{89}{332} \times \frac{109}{332} = 0.0880$$

$$p_e = 0.4916 + 0.0880 = 0.5796$$

$$\kappa = \frac{.8012 - .5796}{1 - .5796} = 0.5271$$

I'll let you read about the other measures given here if interested.

<https://www.dataschool.io/simple-guide-to-confusion-matrix-terminology/>

Receiver Operating Characteristic Curve

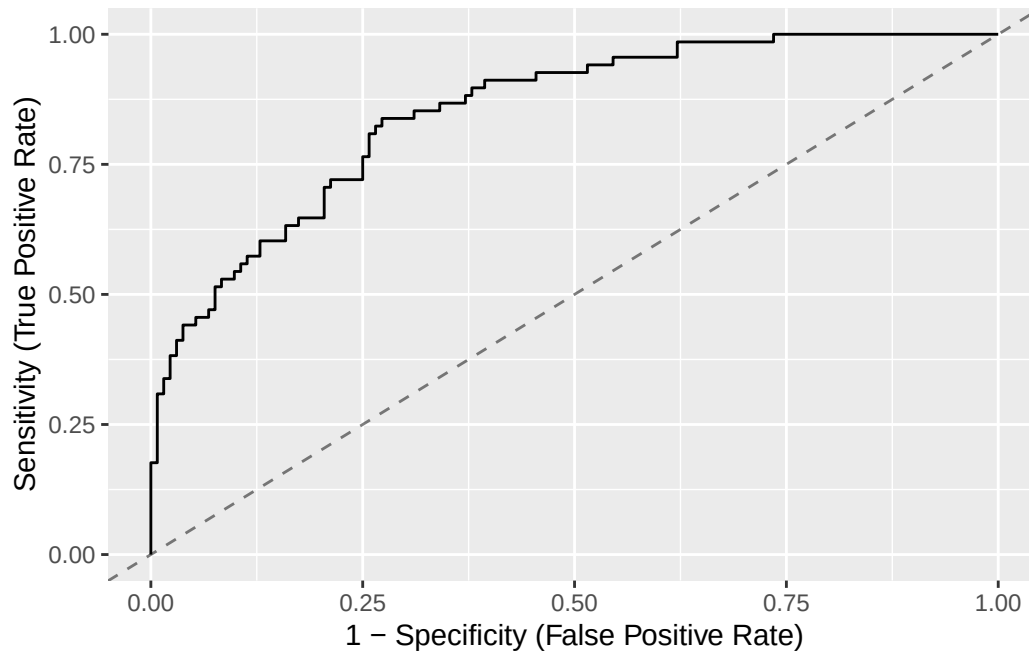
In the words of Ben Baumer, Danny Kaplan, and Nick Horton:

a more principled approach to assessing the quality of a classifier is a *receiver operating characteristic (ROC)* curve. This considers all possible thresholds for rounding, and graphically displays the trade-off between *sensitivity* (the true positive rate) and *specificity* (the true negative rate). What is actually plotted is the true positive rate as a function of the false positive rate.

The area under the *ROC* curve, *AUC* represents a probability that the model/classifier, if give a random positive example (in this example, someone with diabetes) and a random negative example (someone without diabetes), will rank the positive example higher than the negative example. *AUC* = 1 represents perfect classification, with *AUC* = 0.5 chance-level classification.

I am using the `performance` package to compute and graph the *ROC* curve and *AUC* of our logistic regression model. This can be done with many other packages.

```
# maybe replace with yardstick package, roc_curve and roc_auc
# that is in Chapter 10 of Modern Data Science With R
# https://mdsr-book.github.io/mdsr3e/10-modeling.html
require(performance)
require(see)
plot(performance_roc(diabetes.model)) #ROC curve
```



```
performance_accuracy(diabetes.model) #AUC
```

```
# Accuracy of Model Predictions
```

```
Accuracy (95% CI): 81.29% [79.21%, 84.69%]
```

```
Method: Area under Curve
```

The caret Package

Max Kuhn's `caret` package (classification and regression training) package also gives us the ability to compare literally dozens of methods from both classical statistics and machine learning via *LOOCV* or *k*-fold cross-validation. I frankly don't know and have never implemented most of these methods.

Here's his book on it.

<https://topepo.github.io/caret/index.html>

I'm going to use his package to show how to implement cross-validation in both a regression (continuous response) and classification (categorical response) setting.

With a continuous response, the most common measure of prediction error is the *root-mean-square-error* or *RMSE*.

$$RMSE(y, \hat{y}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2}$$

Another common measure is *mean absolute error* MAE which will not penalize large residuals (“misses”) as heavily since an absolute value is used in place of the squared term.

$$MAE(y, \hat{y}) = \sqrt{\frac{1}{n} \sum_{i=1}^n |y - \hat{y}|}$$

The coefficient of determination, R^2 is the proportion of variability in the outcome explained by the model. When fitting a linear regression model, this is just the ratio of the sum of squares regression to the sum of squares total. R^2 will be on the scale $[0, 1]$ with $R^2 = 1$ meaning a perfect match between y and $\hat{y}$.

The `kernlab` package is just for the `spam` data set from the books, as the `ElemStatLearn` package is no longer available on CRAN.

```
require(tidyverse)
require(MASS)
require(caret)

## Regression Setting, compare multiple linear regression (lm) with CART (rpart)
## use complete Pima data set Pima.tr2

# set up Leave-One-Out CV
train_controlLOOCV <- trainControl(method="LOOCV",
                                   classProbs=FALSE)

# train the model using linear regression
model.linreg <- train(bmi~.,
                     data=na.omit(Pima.tr2),
                     trControl=train_controlLOOCV,
                     method="lm")
print(model.linreg)
```

Linear Regression

200 samples

7 predictor

No pre-processing

Resampling: Leave-One-Out Cross-Validation

Summary of sample sizes: 199, 199, 199, 199, 199, 199, ...

Resampling results:

RMSE	Rsquared	MAE
4.765348	0.3999993	3.595549

Tuning parameter 'intercept' was held constant at a value of TRUE

```
# train the model using cart
model.cart <- train(bmi~.,
                    data=na.omit(Pima.tr2),
                    trControl=train_controlLOOCV,
                    method="rpart")

print(model.cart)
```

CART

200 samples

7 predictor

No pre-processing

Resampling: Leave-One-Out Cross-Validation

Summary of sample sizes: 199, 199, 199, 199, 199, 199, ...

Resampling results across tuning parameters:

cp	RMSE	Rsquared	MAE
0.03557495	4.705115	0.415485233	3.790701
0.09024525	5.187052	0.296088970	4.312051
0.41267800	6.654030	0.002905366	5.898693

RMSE was used to select the optimal model using the smallest value.

The final value used for the model was cp = 0.03557495.

When I used Leave-One-Out cross-validation comparing linear regression and CART, the results are very similar. Since we would wish to minimize RMSE, the slightly lower value for the CART model is desirable, although you then get to dig into what the **cp** (model complexity) tuning parameter means. If you like R^2 better, it's slightly higher (more desirable) with CART than linear regression.

Let's try for a classification problem, comparing logistic regression and CART for the **spam** vs **ham** example (i.e. is an email legitimate or spam)?

```

require(tidyverse)
require(kernlab) # just for the spam data set
require(caret)

## Classification setting, compare logistic regression (glm) with CART (rpart)
## use complete Pima data set spam

# set up k-folds with k=10
train_controlKFCV <- trainControl(method="cv",
                                  number=10,
                                  classProbs=TRUE)

# train the model using linear regression
data(spam)
model.logreg <- train(type~.,
                      data=spam,
                      trControl=train_controlKFCV,
                      method="glm")
print(model.logreg)

```

Generalized Linear Model

4601 samples
 57 predictor
 2 classes: 'nonspam', 'spam'

No pre-processing

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 4140, 4141, 4141, 4140, 4142, 4141, ...

Resampling results:

Accuracy	Kappa
0.9269748	0.8460492

```

# train the model using cart
model.cart <- train(type~.,
                    data=spam,
                    trControl=train_controlKFCV,
                    method="rpart")

print(model.cart)

```

CART

```
4601 samples
  57 predictor
  2 classes: 'nonspam', 'spam'
```

No pre-processing

Resampling: Cross-Validated (10 fold)

Summary of sample sizes: 4141, 4142, 4141, 4140, 4141, 4140, ...

Resampling results across tuning parameters:

cp	Accuracy	Kappa
0.04302261	0.8559095	0.6938744
0.14892443	0.7900521	0.5555638
0.47655819	0.6863549	0.2471109

Accuracy was used to select the optimal model using the largest value.

The final value used for the model was cp = 0.04302261.

This time, the data modeling approach (logistic regression, a type of generalized linear model) outdid CART, as both Accuracy and Kappa are statistics that we wish to maximize.

Resources

- Baumer, B, Kaplan, D., & Horton, N. (2024) Modern data science with R (3rd edition). Chapman Hall/CRC Press. Available at <https://mdsr-book.github.io/mdsr3e/>
- Efron, B., & Hastie, T. (2016). Computer age statistical inference. Cambridge University Press. (Look up on Google Scholar for a link to the book)
- Frick, H., Chow, F., Kuhn, M., Mahoney, M., Silge, J., & Wickham, H. (2026). *rsample: General Resampling Infrastructure*. doi:10.32614/CRAN.package.rsample <https://doi.org/10.32614/CRAN.package.rsample>, R package version 1.3.2, <https://CRAN.R-project.org/package=rsample>.
- Gelman, A., Hill, J., & Vehtari, A. (2021). Regression and other stories. Cambridge University Press.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An introduction to statistical learning with applications in R (2nd edition). Springer. Available at <https://www.statlearning.com/>.
- Kuhn, M. (2008). Building Predictive Models in R Using the *caret* Package. Journal of Statistical Software, 28(5), 1–26. <https://doi.org/10.18637/jss.v028.i05>.